Algoritmo HHL

Genesis Silesky

21 de octubre de 2026

1. Algoritmo HHL: resolver un sistema lineal con dos qubits

1.1. Objetivo

Entender cómo el algoritmo HHL resuelve $A\mathbf{x} = \mathbf{b}$ dividiendo los coeficientes de $\mathbf{b}$ entre los autovalores de A, y construir su versión de dos qubits para una matriz 2×2.

1.2. Conceptos previos

Concepto previo: autovalores, autovectores y estimación de fase

Ver capítulo 17 (estimación de fase) y la sección **??**. Una matriz hermítica A se escribe como $A = \sum_i \lambda_i |u_i\rangle \langle u_i|$, con autovalores λ_i y autovectores $|u_i\rangle$.

1.3. Fundamento teórico

HHL (Harrow, Hassidim y Lloyd, 2009) resuelve sistemas de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$, con A una matriz hermítica $N \times N$. Clásicamente, con métodos directos o iterativos, el tiempo es $O(N^3)$; en condiciones ideales, HHL lo reduce a $O(\log N)$. Se usa como subrutina en modelos de aprendizaje automático cuántico.

Un ejemplo clásico. Dos grupos hacen vasijas. En 2.5 horas, el grupo A hace una vasija más que el grupo B en 0.5 horas; y en 0.5 horas, A hace tantas como B en 2.5 horas. Si x e y son las vasijas por hora:

$$\begin{pmatrix} 2.5 & -0.5 \\ -0.5 & 2.5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad x = \tfrac{5}{12},\ y = \tfrac{1}{12}. \tag{1}$$

La idea de HHL. Los vectores se codifican como estados cuánticos normalizados, $|b\rangle$ y $|x\rangle$, y resolver $\mathbf{x} = A^{-1}\mathbf{b}$ pasa a ser $|x\rangle \propto A^{-1}|b\rangle$. Se escribe $|b\rangle$ en la base de autovectores, $|b\rangle = \sum_i \beta_i |u_i\rangle$ con $\beta_i = \langle u_i|b\rangle$, y entonces

$$|x\rangle \ \propto \ \sum_i \frac{\beta_i}{\lambda_i} |u_i\rangle \, . \tag{2}$$

Basta dividir cada coeficiente entre su autovalor. Para la matriz del ejemplo, los autovalores son $\lambda_1 = 2$ y $\lambda_2 = 3$, con autovectores $|u_1\rangle = \frac{1}{\sqrt{2}}\binom{1}{1}$ y $|u_2\rangle = \frac{1}{\sqrt{2}}\binom{1}{-1}$.

Los tres componentes. HHL usa tres grupos de qubits: de codificación (guardan $|b\rangle$, $n_b = \log_2 N$ qubits), de registro (guardan los autovalores estimados) y un qubit auxiliar. Los pasos son:

1. **Estimación de fase:** estima los autovalores de A y los guarda en el registro: $\sum_i \beta_i \left|\tilde{\lambda}_i\right\rangle |u_i\rangle$.

2. **Rotación controlada:** usando el registro como control, gira el qubit auxiliar un ángulo $\theta_i = 2\sin^{-1}(c/\lambda_i)$. Así el recíproco del autovalor pasa a la amplitud del auxiliar: $\sum_i \beta_i |\lambda_i\rangle |u_i\rangle \left(\sqrt{1 - c^2/\lambda_i^2}\,|0\rangle + \frac{c}{\lambda_i}|1\rangle\right)$. El parámetro c cumple $c \leq \text{mín}\,|\lambda_i|$.

3. **Estimación de fase inversa:** desentrelaza el registro.

4. **Medición:** si el auxiliar vale $|1\rangle$ (con cierta probabilidad), el qubit de codificación queda en $|x\rangle \propto \sum_i \frac{\beta_i}{\lambda_i}|u_i\rangle$.

Versión de dos qubits. Con una matriz 2×2 de autovalores 2 y 3, el registro necesita un solo qubit y el circuito se simplifica hasta quedar en dos qubits: un qubit de codificación y uno auxiliar. Si $A = U\,\text{diag}(\lambda_1, \lambda_2)\,U^\dagger$, el circuito es: $U^\dagger$ sobre el qubit de codificación (los autovectores pasan a $|0\rangle$ y $|1\rangle$), una rotación $R_y(\theta_i)$ del auxiliar controlada por el qubit de codificación y, al final, U. Con $\lambda_1 = 2$ y $\lambda_2 = 3$, el mayor valor posible de c es 2.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 18 - Algoritmo HHL: resolver un sistema lineal Ax = b con dos qubits

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# %% Celda 2: el problema clasico
# Dos grupos hacen vasijas: 2.5 x - 0.5 y = 1  y  -0.5 x + 2.5 y = 0.
# En forma matricial es  A x = b  con
A = np.array([[2.5, -0.5], [-0.5, 2.5]])
b = np.array([1.0, 0.0])
x_clasico = np.linalg.solve(A, b)
print("Solucion clasica x = A^-1 b =", x_clasico.round(4), "(5/12, 1/12)")

# %% Celda 3: la idea de HHL, con autovalores y autovectores
# A se escribe con sus autovalores l_i y autovectores |u_i>:  A = suma l_i |u_i><u_i|.
# El vector b se escribe en esa base:  |b> = suma beta_i |u_i>,  beta_i = <u_i|b>.
# Entonces la solucion es  |x> proporcional a  suma (beta_i / l_i) |u_i>:
# basta dividir cada coeficiente entre su autovalor.
autovalores, U = np.linalg.eigh(A)      # las columnas de U son los autovectores
b_norm = b / np.linalg.norm(b)
beta = U.T @ b_norm
x_por_autovalores = U @ (beta / autovalores)
print("Autovalores:", autovalores)
print("beta_i = <u_i|b>:", beta.round(4))
print("Solucion con autovalores (normalizada):", (x_por_autovalores / np.linalg.norm(
    ↪ x_por_autovalores)).round(4))
print("Solucion clasica       (normalizada):", (x_clasico / np.linalg.norm(x_clasico)).round(4))


# %% Celda 4: el circuito de HHL de dos qubits
# HHL tiene tres partes: estimacion de fase (para hallar los autovalores), una
# rotacion controlada (para dividir entre ellos) y la estimacion de fase inversa.
# Para una matriz 2x2 con autovalores conocidos, el circuito se simplifica a dos
# qubits (como en el PDF): q0 = qubit de codificacion, q1 = qubit auxiliar.
#   1. Codificar |b> en q0.
#   2. U^dagger: lleva los autovectores |u_i> a |0> y |1>.
#   3. Rotacion controlada: si q0 vale |i>, Ry(theta_i) sobre q1, con
#      theta_i = 2 arcsin(c / l_i). Asi la amplitud de |1> en q1 es c / l_i.
#   4. U: regresa a la base original.
# c es un numero ajustable con c <= l_min. Aqui se toma el mayor posible.
```

```python
def hhl_2x2(A, b):
    autovalores, U = np.linalg.eigh(A)
    c = autovalores.min()
    qc = QuantumCircuit(2)
    qc.prepare_state(b / np.linalg.norm(b), [0])        # 1. |b> en q0
    qc.unitary(U.conj().T, [0], label="U^dagger")        # 2. cambio de base
    for k, l in enumerate(autovalores):                  # 3. rotaciones controladas
        theta = 2 * np.arcsin(c / l)
        if k == 0:
            qc.x(0)                                      # controla cuando q0 = |0>
        qc.cry(theta, 0, 1)
        if k == 0:
            qc.x(0)
    qc.unitary(U, [0], label="U")                        # 4. vuelta a la base original
    return qc, c


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
hhl_2x2(A, b)[0].draw()


# %% Celda 5: ejecutar y quedarse con el caso "auxiliar = 1"
# La solucion aparece en q0 solo cuando q1 se mide en |1>. Se obtiene con
# probabilidad P1 (depende de c). Las amplitudes de q0 dadas q1 = 1 son proporcionales a x.
def resolver(A, b):
    qc, c = hhl_2x2(A, b)
    amplitudes = Statevector(qc).data           # indices: q1 q0 = 00, 01, 10, 11
    cond = amplitudes[2:]                        # q1 = 1: q0 = 0, 1
    prob_exito = float(np.sum(abs(cond) ** 2))
    return np.real(cond / np.sqrt(prob_exito)), prob_exito


x_cuantico, p_exito = resolver(A, b)
# Una fase global (signo) no importa: se compara el valor absoluto.
x_ref = x_clasico / np.linalg.norm(x_clasico)
print("x cuantico (normalizado):", np.abs(x_cuantico).round(4), "| clasico:", np.abs(x_ref).
    ↪ round(4))
print("Probabilidad de que el auxiliar valga 1:", round(p_exito, 4))

# %% Celda 6: la matriz del experimento
# En el laboratorio se resuelve  A x = b  con
#     A = [[2.14645, -0.35355], [-0.35355, 2.85355]],  b = (0.70711, 0.70711).
# Tiene autovalores 2 y 3. Sus autovectores son (cos(pi/8), sin(pi/8)) y
# (-sin(pi/8), cos(pi/8)): la matriz U es una rotacion Ry(pi/4).
A2 = np.array([[2.14645, -0.35355], [-0.35355, 2.85355]])
b2 = np.array([0.70711, 0.70711])
autovalores2, U2 = np.linalg.eigh(A2)
Ry_pi_4 = np.array([[np.cos(np.pi / 8), -np.sin(np.pi / 8)], [np.sin(np.pi / 8), np.cos(np.pi /
    ↪ 8)]])
print("Autovalores de A:", autovalores2.round(4))
print("U coincide con Ry(pi/4) (salvo signos):", np.allclose(abs(U2), abs(Ry_pi_4), atol=1e-4))

x2, p2 = resolver(A2, b2)
x2_clasico = np.linalg.solve(A2, b2)
x2_clasico = x2_clasico / np.linalg.norm(x2_clasico)
print("x cuantico (normalizado):", np.abs(x2).round(4), "| clasico:", np.abs(x2_clasico).round
    ↪ (4))
print("Probabilidad de que el auxiliar valga 1:", round(p2, 4))

# %% Celda 7: probabilidades que se medirian
# En un equipo real se mide en la base computacional. Las probabilidades de
# |q1 q0> = |00>, |01>, |10>, |11> salen de las amplitudes del estado final.
qc2, _ = hhl_2x2(A2, b2)
```

```python
probs = np.abs(Statevector(qc2).data) ** 2
for etiqueta, p in zip(("00", "01", "10", "11"), probs):
    print(f"P(|{etiqueta}>) = {p:.4f}")
```

1.5. Explicación celda por celda

Celda 2 Resuelve el sistema de las vasijas de forma clásica.

Celda 3 Calcula autovalores y autovectores, los coeficientes β_i, y comprueba que $\sum_i (\beta_i/\lambda_i)\,|u_i\rangle$ da la misma solución que la clásica (normalizada).

Celda 4 Define el circuito HHL de dos qubits: codificar $|b\rangle$, $U^\dagger$, rotaciones controladas y U.

Celda 5 Calcula las amplitudes del qubit de codificación cuando el auxiliar vale $|1\rangle$ y las compara con la solución clásica.

Celda 6 Repite con la matriz del experimento y comprueba que su U es $R_y(\pi/4)$.

Celda 7 Imprime las probabilidades de las cuatro cadenas $|q_1 q_0\rangle$ que se medirían.

1.6. Salida obtenida

```
Solucion clasica x = A^-1 b = [0.4167 0.0833] (5/12, 1/12)
Autovalores: [2. 3.]
beta_i = <u_i|b>: [-0.7071 -0.7071]
Solucion con autovalores (normalizada): [0.9806 0.1961]
Solucion clasica       (normalizada): [0.9806 0.1961]
x cuantico (normalizado): [0.9806 0.1961] | clasico: [0.9806 0.1961]
Probabilidad de que el auxiliar valga 1: 0.7222
Autovalores de A: [2. 3.]
U coincide con Ry(pi/4) (salvo signos): True
x cuantico (normalizado): [0.7887 0.6148] | clasico: [0.7887 0.6148]
Probabilidad de que el auxiliar valga 1: 0.9186
P(|00>) = 0.0119
P(|01>) = 0.0694
P(|10>) = 0.5714
P(|11>) = 0.3472
```

(El dibujo del circuito no se reproduce aquí.) Es determinista.

1.7. Interpretación

Para el sistema de las vasijas, la solución cuántica normalizada es $(0.9806, 0.1961)$, igual que la clásica $(5/12, 1/12)$ normalizada. Para la matriz del experimento, $A = \left(\begin{smallmatrix} 2.14645 & -0.35355 \\ -0.35355 & 2.85355 \end{smallmatrix}\right)$ y $\mathbf{b} = (0.70711, 0.70711)$, también coinciden: $(0.7887, 0.6148)$. Sus autovalores son 2 y 3 y su matriz U coincide con $R_y(\pi/4)$ (salvo signos). Al medir, la solución solo se obtiene cuando el auxiliar vale 1, con probabilidad 0.72 en el primer caso y 0.92 en el segundo.

Qué se simplifica en este ejemplo

Igual que en el PDF de la práctica, aquí los autovalores de A se *conocen* y se usan directamente para armar las rotaciones; la estimación de fase que los obtiene en un HHL completo se sustituye por esa información previa. Además, un estado cuántico no se puede leer entero: el código calcula las amplitudes con el simulador, lo que un equipo real no permite.

1.8. Ejercicios

1. Cambia $\mathbf{b}$ por $(\cos\varphi, \sin\varphi)$ con otro ángulo y compara con la solución clásica.
2. Cambia c a un valor menor que 2: ¿qué pasa con la probabilidad de que el auxiliar valga 1?
3. Prueba con una matriz 2×2 simétrica distinta (con autovalores positivos) y comprueba el resultado.

1.9. Errores comunes

- Confundir $|x\rangle$ con $\mathbf{x}$: HHL entrega el vector *normalizado*, no su tamaño.
- Elegir c mayor que el menor autovalor: entonces $c/\lambda_i > 1$ y el ángulo $2\sin^{-1}(c/\lambda_i)$ no existe.